

MODUL 10

Design Pattern 1 (Composite – Observer)



Praktikum Pemrograman Berorientasi Objek
Semester Genap 2025-2026
oleh Asisten Dosen Pemrograman Berorientasi Objek

Tujuan :

1. Praktikan memahami operasi Design Pattern Composite dan Observer.
2. Praktikan dapat menerapkan konsep Design Pattern Composite dan Observer terhadap penyelesaian suatu permasalahan.

Dasar Teori :

Design Pattern terbagi menjadi 3 jenis, yaitu Creational Design Pattern, Structural Design Pattern, dan Behavioral Design Pattern. Modul ini tidak akan membahas secara keseluruhan mengenai setiap jenis Design Pattern dan masing-masing penerapannya melainkan hanya berfokus pada Structural Design Pattern terutama pada **Composite** dan Behavioral Design Pattern terutama pada **Observer**.

1. Composite

Composite yang juga dikenal dengan nama Object Tree merupakan salah satu penerapan Structural Design Pattern, composite digunakan saat kita ingin menyatukan banyak objek menjadi satu struktur seperti pohon, lalu memperlakukan semua objek itu (baik yang sederhana maupun yang terdiri dari bagian-bagian lain) dengan cara yang sama.

Komponen Utama Pola Composite:

- Component (Komponen):

Ini adalah antarmuka atau kelas dasar yang digunakan oleh semua objek dalam struktur. Di sinilah ditentukan method umum, misalnya doThis(), yang harus dimiliki oleh semua objek.

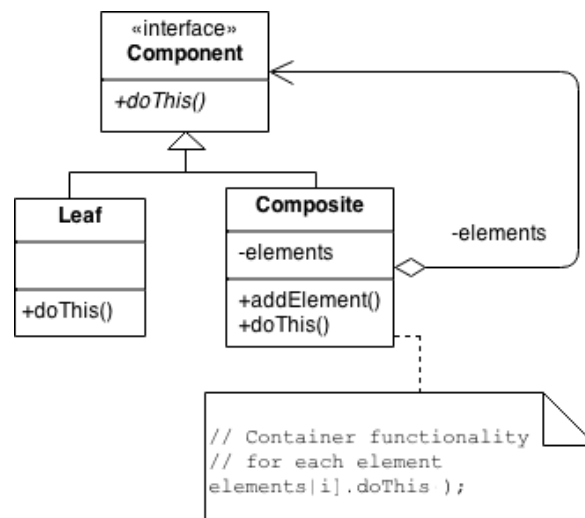
- Leaf (Daun):

Ini adalah objek tunggal yang tidak memiliki anak. Objek ini menjalankan fungsinya sendiri secara langsung.

- Composite (Komposit):

Ini adalah objek yang bisa berisi banyak objek lain, baik Leaf maupun Composite lainnya. Composite juga menjalankan method yang sama (doThis()), tetapi biasanya dengan meneruskan perintah ke semua anak-anaknya.

Ilustrasi sederhana, bayangkan struktur folder di komputer:

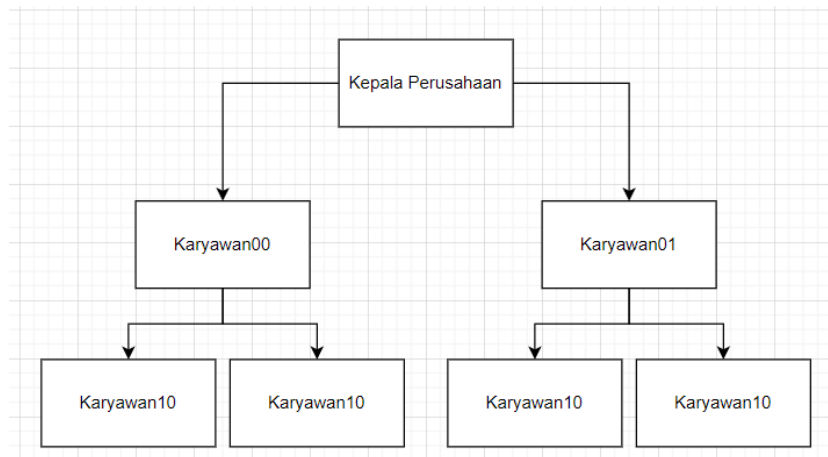


- Component = folder atau file (karena keduanya bisa dibuka)
- Leaf = file (tidak punya isi lain)
- Composite = folder (bisa berisi file dan folder lain)

Kapan Digunakan?

- Saat struktur data Anda berbentuk pohon atau bertingkat (seperti folder, menu, atau bagan organisasi).
- Saat Anda ingin memperlakukan file dan folder (atau item sejenis lainnya) dengan cara yang sama dalam kode.

Salah satu contoh penerapan paling dekat adalah struktur perusahaan. Perusahaan pasti memiliki seorang kepala perusahaan, kepala perusahaan inilah yang digunakan sebagai akar pohon yang akan memiliki cabang-cabang berupa karyawan. Karyawan yang menjabat langsung di bawah kepala perusahaan tentunya tidak hanya berjumlah satu orang, tetapi karyawan tersebut sangat mungkin untuk memiliki karyawan lain untuk membantunya mengerjakan tugas-tugas yang diberikan kepala perusahaan begitu pun seterusnya.



2. Observer

Observer atau yang dikenal juga dengan Event-Subscriber ataupun Listener merupakan salah satu contoh penerapan Behavioral Design Pattern, Observer adalah cara agar satu objek bisa memberitahu banyak objek lainnya ketika terjadi perubahan.

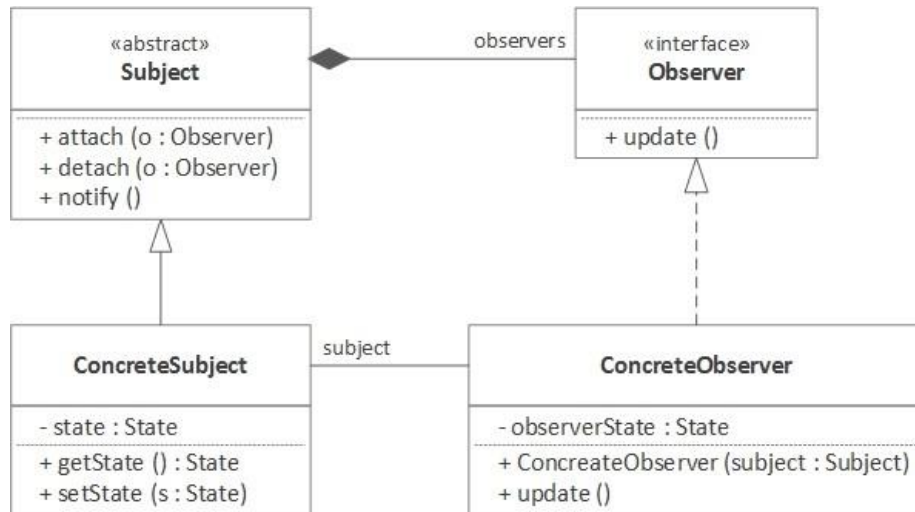
Misalnya:

- Ada Subject (sumber data).
- Ada banyak Observer (pemantau).

Subject ini adalah data yang diamati, saat Subject berubah, semua Observer akan langsung diberi tahu agar bisa ikut berubah atau bereaksi.

Komponen Utama:

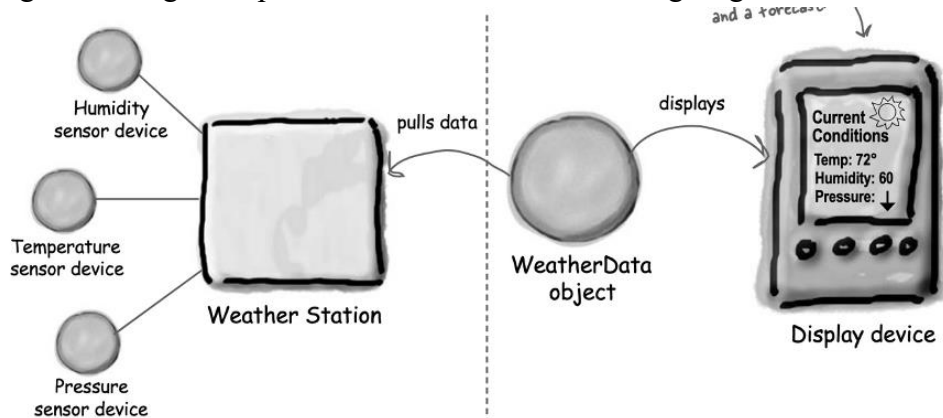
- **Abstract Subject (Subjek):**
Objek utama yang statusnya bisa berubah. Ia menyimpan daftar Observer dan punya method:
 - attach() untuk menambahkan Observer
 - detach() untuk melepas Observer
 - notify() untuk memberi tahu semua Observer saat ada perubahan
- **Abstract Observer (Pengamat):**
Antarmuka (interface) atau kelas abstrak dengan method seperti update(), yang akan dipanggil oleh Subject.
- **Concrete Subject:**
Implementasi nyata dari Subject. Menyimpan data yang bisa berubah, dan saat data ini berubah, semua Observer diberi tahu.
- **Concrete Observer:**
Implementasi nyata dari Observer. Saat update() dipanggil, Observer ini akan melakukan perubahan sesuai kebutuhan (misalnya, memperbarui tampilan, melakukan perhitungan).



Kapan Digunakan?

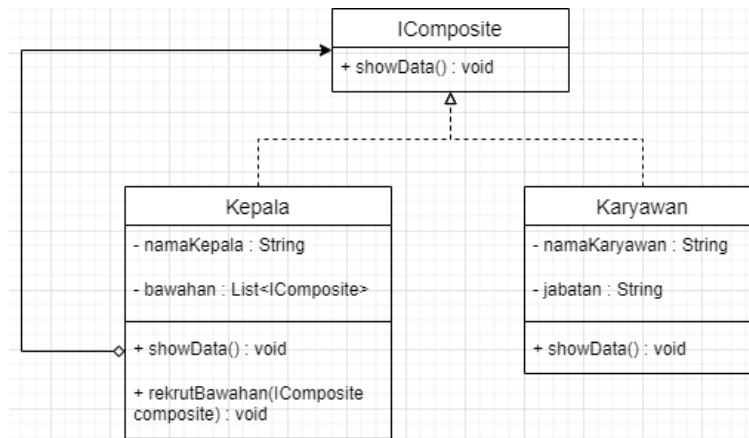
- Saat satu objek berubah dan Anda ingin objek lain otomatis tahu tanpa membuat hubungan yang rumit.
- Saat membuat sistem notifikasi, seperti pembaruan UI saat data berubah.
- Cocok untuk arsitektur MVC, di mana model memberi tahu view.

Salah satu contoh penerapan yang cukup dekat adalah saat kamu membuka aplikasi cuaca, kamu melihat informasi seperti suhu, kelembapan, dan ramalan hujan. Semua informasi ini bergantung pada data utama dari pusat cuaca. Ketika data cuaca diperbarui (misalnya suhu naik), maka semua tampilan akan otomatis ikut berubah tanpa kamu perlu menyegarkan aplikasi secara manual. Ini terjadi karena widget-widget itu mengamati perubahan data cuaca secara langsung.



GUIDED

Pada Guided Design Pattern I ini, praktikan diminta untuk membuat masing-masing program untuk setiap pola sesuai dengan penjelasan teori sebelumnya. Kasus kali ini akan menerapkan Composite untuk membuat sebuah hierarki Perusahaan. Seorang Kepala perusahaan yang bisa memiliki bawahan berupa kepala lain maupun karyawan. Dimana kelas Kepala merupakan Composite, sehingga bisa memiliki bawahan lagi. Sedangkan kelas Karyawan merupakan Leaf, sehingga tidak bisa memiliki bawahan. Diagram kelasnya adalah sebagai berikut.



Buatlah sebuah project baru dengan nama GD10_Y_XXXXX pada aplikasi NetBeans. Kemudian, dilanjutkan dengan membuat interface terlebih dahulu dengan nama **IComposite** yang memiliki isi program sebagai berikut ini.

```
1. /**
2.  *
3.  * @author NPM_Lengkap
4.  */
5. public interface IComposite {
6.     public static StringBuffer space = new StringBuffer();
7.
8.     public void showData();
9. }
10.
```

Space hanya akan digunakan untuk mengatur tata letak tampilan saja, tidak berpengaruh terhadap keseluruhan fungsionalitas program. Setiap kelas yang dibutuhkan untuk menjadi struktur composite ini (dimana pada guided kali ini, kelas tersebut yaitu Karyawan dan Kepala), harus diimplementkan dengan interface **Icomposite**. Selanjutnya membuat kelas karyawan dan kelas kepala dengan masing-masing isi program sebagai berikut.

```
1. /**
2.  *
3.  * @author NPM_Lengkap
4.  */
5. public class Karyawan implements IComposite {
```

```

6.     private String namaKaryawan;
7.     private String jabatan;
8.
9.     public Karyawan(String namaKaryawan, String jabatan) {
10.         this.namaKaryawan = namaKaryawan;
11.         this.jabatan = jabatan;
12.     }
13.
14.     @Override
15.     public void showData() {
16.         System.out.println(namaKaryawan + " (" + jabatan + ")");
17.     }
18. }
19.

```

```

1. import java.util.ArrayList;
2. /**
3.  *
4.  * @author NPM_Lengkap
5.  */
6. public class Kepala implements IComposite {
7.     private String namaKepala;
8.     private ArrayList<IComposite> bawahan;
9.
10.    public Kepala(String namaKepala) {
11.        this.namaKepala = namaKepala;
12.        this.bawahan = new ArrayList<>();
13.    }
14.
15.    public void rekrutBawahan(IComposite composite) {
16.        bawahan.add(composite);
17.    }
18.
19.    @Override
20.    public void showData() {
21.        System.out.println(IComposite.space + "Kepala : " + namaKepala);
22.        IComposite.space.append(" ");
23.
24.        for (IComposite composite : bawahan) {
25.            System.out.print(IComposite.space + "Bawahan dari " + namaKepala + " ");
26.            composite.showData();
27.        }
28.
29.        IComposite.space.setLength(2);
30.    }
31. }
32.

```

Penggunaan `for (Icomposite composite : bawahan)` merupakan contoh penggunaan Range-Based Loop yang sangat mempermudah proses iterasi dalam suatu array, list, ataupun arrayList. Akan tetapi, tetap disarankan untuk menggunakan perulangan `for` atau `while` apabila menggunakan array atau list yang mengandung indeks. Terdapat penggunaan perulangan atau recursive pada bagian `composite.showData()`; yang akan menyesuaikan data yang ada di dalam array atau list. Contoh recursive akan terlihat jelas pada saat program berjalan nantinya.

Selanjutnya, silahkan tambahkan kode berikut ke dalam main class dan jalankan program.

```

1. // Bagian Composite
2.     Kepala direktur = new Kepala("Direktur NAMA PANJANG PRAKTIKAN");
3.     Kepala manager = new Kepala("Manager NAMA PANGGILAN PRAKTIKAN");
4.     Kepala asisten = new Kepala("Asisten Leo");
5.
6.     Karyawan cs = new Karyawan("Kevin", "Customer Service");
7.     Karyawan sekretaris = new Karyawan("Fabian", "Sekretaris");
8.     Karyawan pembantu = new Karyawan("Frendy", "Pembantu Asisten");
9.
10.    direktur.rekrutBawahan(manager);
11.    direktur.rekrutBawahan(asisten);
12.
13.    manager.rekrutBawahan(cs);
14.    manager.rekrutBawahan(sekretaris);
15.
16.    asisten.rekrutBawahan(pembantu);
17.
18.    System.out.println("\n--- Bagian Composite ---");
19.    direktur.showData();
20.

```

Berikut ini merupakan hasil output saat program dijalankan.

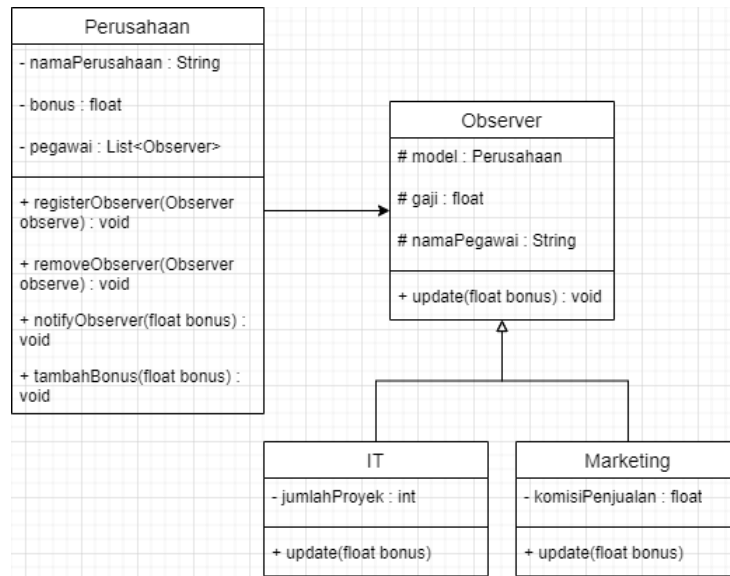
run:

```

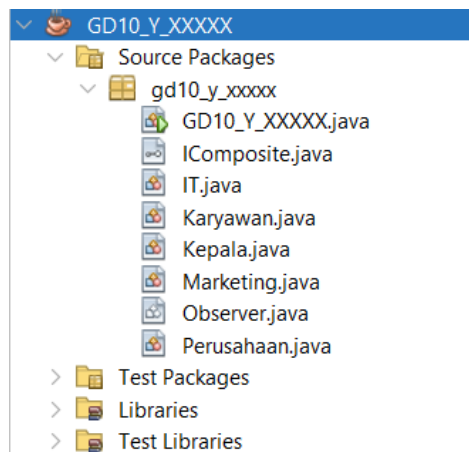
--- Bagian Composite ---
Kepala : Direktur NAMA PANJANG PRAKTIKAN
  Bawahan dari Direktur NAMA PANJANG PRAKTIKAN   Kepala : Manager NAMA PANGGILAN PRAKTIKAN
    Bawahan dari Manager NAMA PANGGILAN PRAKTIKAN Kevin (Customer Service)
    Bawahan dari Manager NAMA PANGGILAN PRAKTIKAN Fabian (Sekretaris)
  Bawahan dari Direktur NAMA PANJANG PRAKTIKAN   Kepala : Asisten Leo
    Bawahan dari Asisten Leo Frendy (Pembantu Asisten)
BUILD SUCCESSFUL (total time: 0 seconds)

```

Kasus berikutnya akan menerapkan Observer pada guided ini yaitu, terdapat dua observer yaitu IT dan Marketing, yang bisa didaftarkan pada class Subjectnya (Perusahaan). Observer tersebut mengamati atribut kelas Perusahaan yang bernama bonus. Ketika terdapat perubahan terhadap variabel bonus melalui fungsi TambahBonus(float bonus), maka semua observer yang mengamati subjek ini akan diberi tahu dan akan menjumlahkan gaji yang lama dengan variabel bonus yang dipengaruhi juga oleh jumlahProjek yang dikerjakan, maupun jumlah komisiPenjualan yang didapat. untuk memberikan bonus pegawai pada masing-masing divisi. Berikut ini adalah tampilan diagram kelasnya.



Silahkan tambah kelas abstrak Observer, kelas Perusahaan, kelas IT dan kelas Marketing berikut ke dalam Guided yang sudah ada sehingga tampilan akhirnya menjadi sebagai berikut ini.



Selanjutnya, silahkan tambahkan kode-kode berikut ke dalam masing-masing kelas.

Kita akan mengisi kode kelas abstrak (Observer) terlebih dahulu.

```

1. /**
2.  *
3.  * @author Nama_Lengkap
4.  */
5. public abstract class Observer {
6.     protected Perusahaan model;
7.     protected String namaPegawai;
8.     protected float gaji;
9. }
  
```

```

10.     public Observer(Perusahaan model, String namaPegawai, float gaji) {
11.         this.model = model;
12.         this.namaPegawai = namaPegawai;
13.         this.gaji = gaji;
14.         this.model.registerObserver(this);
15.     }
16.
17.     public abstract void update(float bonus);
18. }
19.

```

Kemudian kita akan mengisi kedua class yang merupakan kelas anak dari Observer yaitu IT dan Marketing, mari kita isi mulai dari IT.java.

```

1.  /**
2.   *
3.   * @author Nama_Lengkap
4.   */
5.  public class IT extends Observer {
6.      private int jumlahProyek;
7.
8.      public IT(int jumlahProyek, Perusahaan model, String namaPegawai, float gaji) {
9.          super(model, namaPegawai, gaji);
10.         this.jumlahProyek = jumlahProyek;
11.     }
12.
13.     @Override
14.     public void update(float bonus) {
15.         float tempGaji = gaji;
16.         gaji += bonus * (jumlahProyek / 10);
17.         System.out.println(namaPegawai + " -> Gaji Sebelum : " + tempGaji + " -> Gaji Sesudah : " + gaji);
18.     }
19. }
20.

```

Kelas Marketing.java memiliki code yang hampir sama dengan IT, tetapi terdapat sedikit perbedaan pada rumus perhitungan bonusnya.

```

1.  public class Marketing extends Observer {
2.      private float komisiPenjualan;
3.
4.      public Marketing(float komisiPenjualan, Perusahaan model, String namaPegawai, float gaji) {
5.          super(model, namaPegawai, gaji);
6.          this.komisiPenjualan = komisiPenjualan;
7.      }
8.
9.      @Override
10.     public void update(float bonus) {
11.         float tempGaji = gaji;
12.         gaji += bonus + (komisiPenjualan * 1);
13.         System.out.println(namaPegawai + " -> Gaji Sebelum : " + tempGaji + " -> Gaji Sesudah : " + gaji);
14.     }
15. }
16.

```

Kita akan melanjutkan ke kelas subjectnya yaitu Perusahaan.java.

```
1. import java.util.ArrayList;
2. /**
3.  *
4.  * @author Nama_Lengkap
5.  */
6. public class Perusahaan {
7.     private String namaPerusahaan;
8.     private float bonus;
9.     private ArrayList<Observer> pegawai;
10.
11.     public Perusahaan(String namaPerusahaan) {
12.         this.namaPerusahaan = namaPerusahaan;
13.         this.bonus = 0;
14.         this.pegawai = new ArrayList<>();
15.     }
16.
17.     public void registerObserver(Observer observe) {
18.         pegawai.add(observe);
19.     }
20.
21.     public void removeObserver(Observer observe) {
22.         pegawai.remove(observe);
23.     }
24.
25.     public void notifyObserver(float bonus) {
26.         for (Observer observe : pegawai) {
27.             observe.update(bonus);
28.         }
29.     }
30.
31.     public void tambahBonus (float bonus) {
32.         this.bonus = bonus;
33.         notifyObserver(bonus);
34.     }
35. }
36
```

Selanjutnya tambahkan kode di dalam main class sehingga tampak seperti di bawah ini.

```
1. /**
2.  *
3.  * @author NPM_Lengkap
4.  */
5. public class GD10_Y_XXXXX {
6.
7.     /**
8.      * @param args the command line arguments
9.      */
10.    public static void main(String[] args) {
11.        // TODO code application logic here
12.
13.        // Bagian Observer
14.        Perusahaan perusahaan = new Perusahaan("Perusahaan Atma");
15.
16.        Observer pegawai1 = new Marketing(30000, perusahaan, "Padmayoga", 10000);
17.        Observer pegawai2 = new IT(25000, perusahaan, "Riztianda", 8000);
18.
19.        System.out.println("--- Bagian Observer ---");
20.        perusahaan.tambahBonus(2000);
21.        perusahaan.removeObserver(pegawai2);
22.        System.out.println("Setelah Remove Pegawai IT");
23.        perusahaan.tambahBonus(6000);
24.        perusahaan.removeObserver(pegawai1);
25.
26.        // Bagian Composite
27.        Kepala direktur = new Kepala("Direktur NAMA PANJANG PRAKTIKAN");
28.        Kepala manager = new Kepala("Manager NAMA PANGGILAN PRAKTIKAN");
29.        Kepala asisten = new Kepala("Asisten Leo");
30.
31.        Karyawan cs = new Karyawan("Kevin", "Customer Service");
32.        Karyawan sekretaris = new Karyawan("Fabian", "Sekretaris");
33.        Karyawan pembantu = new Karyawan("Frendy", "Pembantu Asisten");
34.
35.        direktur.rekrutBawahan(manager);
36.        direktur.rekrutBawahan(asisten);
37.
38.        manager.rekrutBawahan(cs);
39.        manager.rekrutBawahan(sekretaris);
40.
41.        asisten.rekrutBawahan(pembantu);
42.
43.        System.out.println("\n--- Bagian Composite ---");
44.        direktur.showData();
45.    }
46.
47. }
48.
```

Hasil dari output dari program tersebut akan terlihat seperti pada lampiran di bawah ini.

```
run:
--- Bagian Composite ---
Padmayoga -> Gaji Sebelum : 10000.0 -> Gaji Sesudah : 42000.0
Riztianda -> Gaji Sebelum : 8000.0 -> Gaji Sesudah : 5008000.0
Setelah Remove Pegawai IT
Padmayoga -> Gaji Sebelum : 42000.0 -> Gaji Sesudah : 78000.0

--- Bagian Composite ---
Kepala : Direktur NAMA PANJANG PRAKTIKAN
  Bawahan dari Direktur NAMA PANJANG PRAKTIKAN   Kepala : Manager NAMA PANGGILAN PRAKTIKAN
    Bawahan dari Manager NAMA PANGGILAN PRAKTIKAN Kevin (Customer Service)
    Bawahan dari Manager NAMA PANGGILAN PRAKTIKAN Fabian (Sekretaris)
  Bawahan dari Direktur NAMA PANJANG PRAKTIKAN   Kepala : Asisten Leo
    Bawahan dari Asisten Leo Frendy (Pembantu Asisten)
BUILD SUCCESSFUL (total time: 0 seconds)
```

Sumber: <https://refactoring.guru/design-patterns>

KETENTUAN GUIDED

1. Penamaan project NetBeans yaitu **GD10_Y_XXXXX** dengan huruf Y merepresentasikan kelas dan huruf X merepresentasikan 5 digit NPM akhir praktikan;
2. Pekerjaan dilakukan pada aplikasi NetBeans yang sudah terinstal pada laptop praktikan masing-masing;
3. Berkas yang dikumpulkan adalah hasil zip langsung dari folder project NetBeans sehingga tampilan Ketika dibuka adalah sebagai berikut;
4. Bonus tambahan nilai UGD sebesar 5 poin akan diberikan kepada praktikan yang berhasil mengumpulkan paling cepat pada masing-masing kelas praktikum;
5. Ketentuan lain tetap mengikuti sesuai dengan ketentuan yang terdapat pada Spreadsheet Praktikum. Perubahan atau penambahan ketentuan akan dikomunikasikan lebih lanjut di dalam kelas praktikum.

